DEBUGGING A BROKEN WORKFLOW USING LANGGRAPH

Assignment – Generative AI

---

1. Introduction

LangGraph is a powerful framework designed to build structured and stateful workflows for applications powered by Large Language Models (LLMs). It enables developers to create multi-step pipelines where each step, referred to as a node, performs a specific operation and passes the result to the next stage through a shared state.

With the increasing adoption of AI-driven systems such as chatbots, virtual assistants, and automated decision-making tools, workflow orchestration has become a critical component. LangGraph simplifies this process by providing a clear architecture for defining execution flow, managing intermediate data, and ensuring consistency across different stages.

However, designing such workflows is not always straightforward. Errors in state management, node connections, or output formats can lead to system failures or incorrect responses. Debugging these workflows requires a systematic approach to identify and resolve issues effectively.

This assignment focuses on analyzing a broken LangGraph workflow, identifying multiple bugs, and implementing the necessary fixes to ensure correct execution. It also highlights the importance of proper workflow design, validation, and testing in real-world AI applications.

---

2. Problem Statement

A LangGraph workflow was designed to process user input through multiple stages:

- Input preprocessing
- Response generation
- Output formatting

However, the workflow contained several intentional bugs that caused execution failure and incorrect outputs.

The goal is to:

- Identify the bugs
- Fix them
- Provide a corrected implementation
- Explain the improvements

---

3. Identified Bugs and Issues

Bug 1: Incorrect State Definition

Issue:
The state was either missing required fields or not defined using a structured format.

Impact:
Nodes were unable to access necessary data, causing runtime errors.

Bug 2: Missing Node Connection

Issue:
One node was not connected to the next step in the workflow.

Impact:
The workflow execution stopped prematurely and did not reach the final output stage.

Bug 3: Incorrect Return Type

Issue:
A node returned a string instead of a dictionary.

Impact:
LangGraph requires all node outputs to be in dictionary format for state updates, leading to failure.

Bug 4: Inconsistent State Key Usage

Issue:
Different nodes used different key names (e.g., input_text vs user_input).

Impact:
This caused missing data errors and incorrect processing.

Bug 5: No Entry or Exit Point Defined

Issue:
The workflow did not clearly define the starting or ending node.

Impact:
Execution could not begin or terminate correctly.

Bug 6: Lack of Error Handling

Issue:
No validation was performed on user input.

Impact:
Invalid or empty input could break the workflow.

---

4. Example of Broken Code (Before Fix)

```
def generate_response(state):
    return "This is a response"   #  Incorrect format


graph.add_node("response_generator", generate_response)


# Missing edge connection
graph.add_edge("input_processor", "response_generator")


# No finish point
```

Problems:

- Returned string instead of dictionary
- Missing connections
- No defined workflow completion

---

5. Corrected LangGraph Implementation

```
from typing import TypedDict
from langgraph.graph import StateGraph


# Step 1: Define State
class WorkflowState(TypedDict):
    user_input: str
    processed_input: str
    response: str
    final_output: str
```

```
# Step 2: Define Nodes

def process_input(state: WorkflowState):
    text = state.get("user_input", "")

    if not text:
        return {"processed_input": "No input provided"}

    return {"processed_input": text.lower()}


def generate_response(state: WorkflowState):
    processed = state["processed_input"]
    reply = f"Processed query: {processed}"
    return {"response": reply}


def format_output(state: WorkflowState):
    result = state["response"]
    formatted = f"Final Output: {result}"
    return {"final_output": formatted}


# Step 3: Build Graph

graph = StateGraph(WorkflowState)

graph.add_node("input_processor", process_input)
graph.add_node("response_generator", generate_response)
graph.add_node("output_formatter", format_output)
```

```
# Proper workflow connections
graph.set_entry_point("input_processor")
graph.add_edge("input_processor", "response_generator")
graph.add_edge("response_generator", "output_formatter")

graph.set_finish_point("output_formatter")

# Compile
app = graph.compile()



# Step 4: Execute

result = app.invoke({"user_input": "LangGraph Debugging Example"})
print(result["final_output"])
```

---

6. Explanation of Fixes

State Management Fix

All required variables are clearly defined in a structured format using TypedDict, ensuring consistency across nodes.

Node Connectivity Fix

Each node is properly connected in sequence, ensuring smooth data flow from input to output.

Return Format Fix

All node functions now return dictionaries, which is required for updating the shared state.

Key Consistency Fix

Uniform key names are used throughout the workflow, preventing missing data errors.

Entry and Exit Points Fix

The workflow clearly defines where execution starts and ends.

Error Handling Improvement

Basic validation is added to handle empty inputs, improving robustness.

---

7. Workflow Diagram

Logical Flow Representation

User Input

↓

Input Processor

↓

Response Generator

↓

Output Formatter

↓

Final Output

Diagram Instructions for Word

- Use rectangular boxes for each node
- Connect using arrows
- Label each step clearly
- Align vertically for better readability

---

8. Sample Execution

Input:
LangGraph Debugging Example

Output:
Final Output: Processed query: langgraph debugging example

---

9. Learning Outcomes

- Gained understanding of LangGraph workflow structure
- Learned how to debug node-based systems
- Understood importance of state consistency
- Identified common issues in AI workflow pipelines

---

10. Real-World Applications

This type of workflow is commonly used in:

- Customer support chatbots
- Automated query processing systems
- AI-based assistants
- Multi-step data processing pipelines

---

11. Best Practices for Designing LangGraph Workflows

To avoid common errors and improve workflow reliability, the following best practices should be followed while designing LangGraph-based systems:

1. Define a Clear State Structure

- Always define a structured state using TypedDict or similar methods
- Ensure all required variables are included
- Maintain consistent naming conventions across nodes

---

2. Maintain Proper Node Connectivity

- Ensure every node is connected logically
- Avoid isolated or unreachable nodes
- Clearly define entry and exit points

---

3. Use Consistent Return Formats

- Always return outputs as dictionaries
- Match keys with the defined state structure
- Avoid returning raw strings or unsupported formats

---

4. Implement Input Validation

- Validate user inputs before processing
- Handle empty or invalid inputs gracefully
- Prevent runtime errors through basic checks

---

5. Add Logging and Debugging Support

- Print intermediate outputs for debugging
- Track state changes across nodes
- Helps in identifying errors quickly

---

6. Keep Workflows Modular

- Break complex tasks into smaller nodes
- Improves readability and maintainability
- Makes debugging easier

---

7. Test Each Node Independently

- Verify individual node functionality
- Then test the complete workflow
- Ensures reliability at every stage

12. Conclusion

Debugging plays a critical role in ensuring the reliability of AI workflows. Even minor issues such as incorrect return formats or missing connections can disrupt the entire system.

By identifying and fixing these bugs, the workflow becomes more stable, efficient, and scalable. Proper design, validation, and testing are essential when working with frameworks like LangGraph.